Python Activity-Based Learning

Complete report for 10 selected case studies

Includes problem statements, objectives, assumptions, algorithms, flowcharts, pseudocode, Python source code, test cases with captured program output, and conclusions.

The examples use variables, data types, operators, conditions, loops, strings, lists, nested lists, tuples, and tuple unpacking. Policy details not specified in the source list are identified as assumptions.

1. University Scholarship Allocation

Problem statement

Assess scholarship applicants using aggregate marks, subject minimums, attendance, and family income; rank qualified applicants and award a fixed quota.

Objectives

• Apply all eligibility conditions to each applicant.

• Rank qualified applicants using deterministic tie-break rules.

• Select recipients within the quota and explain rejections.

Assumptions

• Eligibility: aggregate >=80, every subject >=60, attendance >=75%, annual income <=Rs.300,000.

• Rank by aggregate descending, attendance descending, then income ascending. Quota is 2.

• Aggregate is supplied; marks and percentages are on a 0..100 scale.

Algorithm

1. Read applicant records and quota.

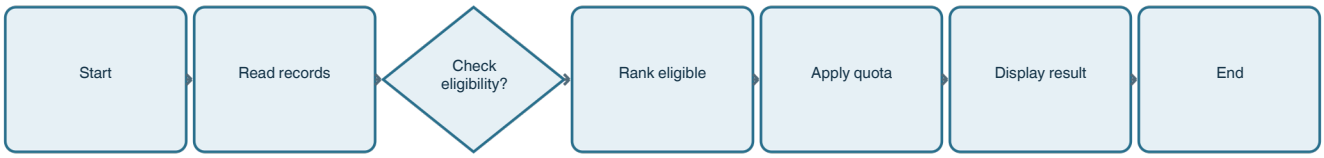
2. Check aggregate, all subject scores, attendance, and income.

3. Store eligible applicants and record reasons for failures.

4. Rank eligible applicants using all tie-break rules.

5. Display awards and rejection reasons.

Flowchart



Pseudocode

```
START
FOR each applicant: check every eligibility rule
IF all pass: add to eligible list ELSE: report failure
ORDER by aggregate DESC, attendance DESC, income ASC
DISPLAY first 2 applicants as awardees
END
```

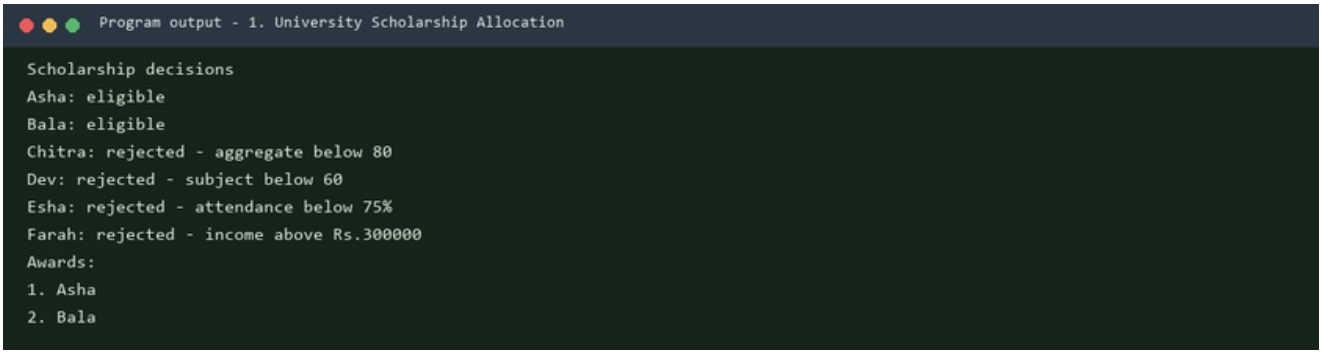
Python source code

```
applicants = [
    ("Asha", 88, [82, 91], 90, 240000), ("Bala", 84, [70, 78], 80, 180000),
    ("Chitra", 79, [90, 85], 92, 120000), ("Dev", 92, [59, 98], 95, 100000),
    ("Esha", 85, [80, 90], 74, 100000), ("Farah", 90, [88, 92], 88, 300001)]
quota = 2
eligible = []
print("Scholarship decisions")
for name, aggregate, subjects, attendance, income in applicants:
    reason = ""
    if aggregate < 80: reason = "aggregate below 80"
    elif min(subjects) < 60: reason = "subject below 60"
    elif attendance < 75: reason = "attendance below 75%"
    elif income > 300000: reason = "income above Rs.300000"
    else: eligible.append((name, aggregate, attendance, income))
    print(name + ": " + ("eligible" if reason == "" else "rejected - " + reason))
for i in range(len(eligible)):
    for j in range(i + 1, len(eligible)):
        a, b = eligible[i], eligible[j]
        if b[1] > a[1] or (b[1] == a[1] and b[2] > a[2]) or (b[1] == a[1] and b[2] == a[2] and b[3] < a[3]):
            eligible[i], eligible[j] = eligible[j], eligible[i]
print("Awards:")
for i in range(min(quota, len(eligible))):
    print(str(i + 1) + ". " + eligible[i][0])
```

Test cases

Test	Input	Expected result
TC1	Asha 88, Bala 84, Chitra 79; other criteria satisfy thresholds.	Asha and Bala qualify and receive ranks 1 and 2; Chitra fails aggregate.
TC2	Dev aggregate 92; subjects 59 and 98.	Reject: one subject is below 60.
TC3	Esha aggregate 85; attendance 74%.	Reject: attendance below 75%.
TC4	Farah aggregate 90; income Rs.300,001.	Reject: income above limit.

Captured program output



Conclusion

Eligibility is checked before ranking, so only qualified applicants compete for the quota. Explicit tie-breaks make awards reproducible.

2. Progressive Electricity Billing

Problem statement

Calculate bills using progressive consumption slabs, fixed charges, a surcharge, and a rebate.

Objectives

• Calculate energy charges progressively across slabs.

• Apply fixed charges, surcharge, and conditional rebate in order.

• Reject invalid negative consumption.

Assumptions

• First 100 kWh Rs.2/unit, next 100 Rs.3/unit, above 200 Rs.5/unit.

• Fixed charge Rs.50; surcharge 5% of energy charge; rebate Rs.20 when usage <=100 kWh.

• Apply rebate after surcharge; bill cannot be below zero.

Algorithm

1. Read each monthly unit value.

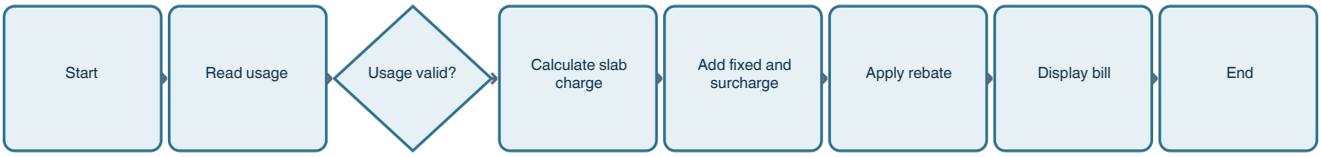
2. Reject negative consumption.

3. Calculate each slab portion and energy charge.

4. Add fixed fee and surcharge; apply rebate when eligible.

5. Display itemized result.

Flowchart



Pseudocode

```
START
FOR each usage value
IF negative: report invalid
ELSE: calculate each slab charge
Add fixed charge and surcharge
IF usage <=100: subtract rebate
DISPLAY bill
END
```

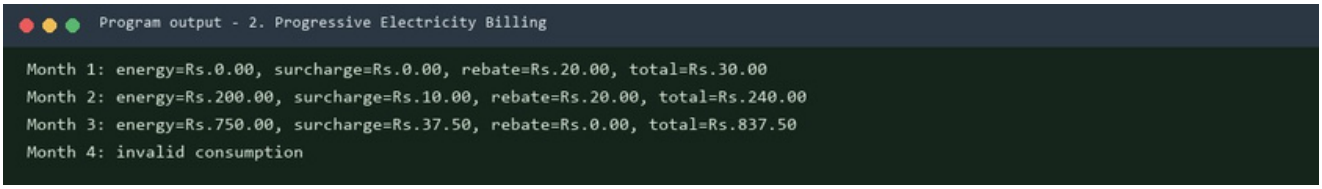
Python source code

```
units_list = [0, 100, 250, -1]
for month, units in enumerate(units_list, 1):
    print("Month " + str(month) + ":", end=" ")
    if units < 0:
        print("invalid consumption")
    else:
        if units <= 100: energy = units * 2
        elif units <= 200: energy = 200 + (units - 100) * 3
        else: energy = 500 + (units - 200) * 5
        surcharge = energy * 0.05
        rebate = 20 if units <= 100 else 0
        bill = max(0, energy + surcharge + 50 - rebate)
        print("energy=Rs." + format(energy, ".2f") + ", surcharge=Rs." +
              format(surcharge, ".2f") + ", rebate=Rs." + format(rebate, ".2f") +
              ", total=Rs." + format(bill, ".2f"))
```

Test cases

Test	Input	Expected result
TC1	0 kWh	Energy Rs.0; fixed Rs.50; rebate Rs.20; bill Rs.30.
TC2	100 kWh	Energy Rs.200; surcharge Rs.10; rebate Rs.20; bill Rs.240.
TC3	250 kWh	Energy Rs.750; surcharge Rs.37.50; fixed Rs.50; bill Rs.837.50.
TC4	-1 kWh	Rejected as invalid negative consumption.

Captured program output



Conclusion

Progressive slab calculation applies the right rate to each portion of consumption. Boundary tests verify the rebate and validation rules.

3. Employee Payroll and Deduction Analysis

Problem statement

Calculate gross salary from basic pay, allowances, overtime, and incentives, apply progressive deductions, and display net pay.

Objectives

• Combine salary components into gross pay.

• Apply deductions in progressive income bands.

• Validate nonnegative inputs and calculate take-home pay.

Assumptions

• Gross = basic + allowances + overtime hours x hourly rate + incentive.

• Deduct 0% on first Rs.20,000; 10% on next Rs.20,000; 20% above Rs.40,000.

• All inputs are nonnegative; display currency to two decimals.

Algorithm

1. Read employee pay components.

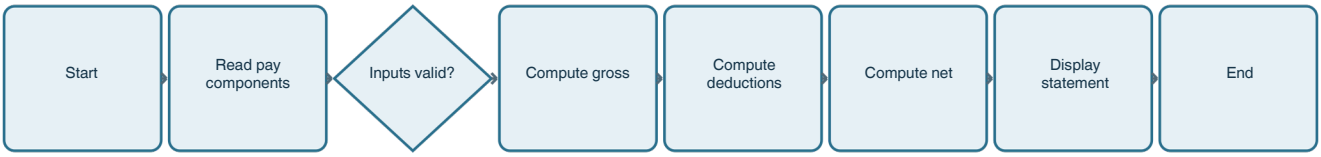
2. Validate inputs.

3. Calculate gross pay.

4. Calculate deductions for each applicable band.

5. Subtract deductions and display the salary statement.

Flowchart



Pseudocode

```
START
FOR each payroll record
IF any input negative: report invalid
ELSE calculate gross
Calculate 10% band up to Rs.40,000
Calculate 20% amount above Rs.40,000
DISPLAY gross, deduction, net
END
```

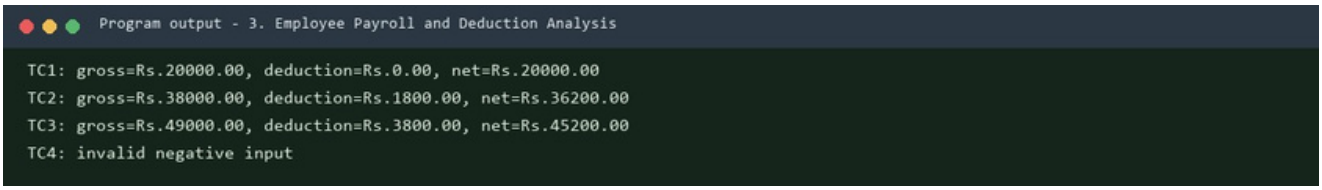
Python source code

```
  records = [("TC1",18000,2000,0,200,0),("TC2",30000,5000,10,200,1000),
             ("TC3",40000,5000,10,200,2000),("TC4",25000,-100,2,200,0)]
for label, basic, allowance, hours, rate, incentive in records:
    print(label + ":", end=" ")
    if min(basic, allowance, hours, rate, incentive) < 0:
        print("invalid negative input")
    else:
        gross = basic + allowance + hours * rate + incentive
        deduction = 0
        if gross > 20000: deduction += (min(gross, 40000) - 20000) * 0.10
        if gross > 40000: deduction += (gross - 40000) * 0.20
        print("gross=Rs." + format(gross, ".2f") + ", deduction=Rs." +
              format(deduction, ".2f") + ", net=Rs." + format(gross-deduction, ".2f"))
```

Test cases

Test	Input	Expected result
TC1	Basic 18,000; allowance 2,000; no overtime or incentive.	Gross/net Rs.20,000; deduction Rs.0.
TC2	Basic 30,000; allowance 5,000; 10 overtime hours at Rs.200; incentive 1,000.	Gross Rs.38,000; deduction Rs.1,800; net Rs.36,200.
TC3	Basic 40,000; allowance 5,000; 10 overtime hours at Rs.200; incentive 2,000.	Gross Rs.49,000; deduction Rs.3,800; net Rs.45,200.
TC4	Negative allowance Rs.100.	Rejected as invalid.

Captured program output



Conclusion

Separating gross pay from progressive deductions produces a transparent salary statement and correctly handles deduction bands.

4. Retail Billing with Multiple Discount Rules

Problem statement

Process products and quantities, apply category-specific taxes and a capped threshold discount, and generate an itemized invoice.

Objectives

• Calculate subtotal and discount for a product basket.

• Apply a discount cap and category tax rates.

• Validate categories and positive whole-number quantities.

Assumptions

• Groceries tax 5%, electronics tax 18%.

• At subtotal >=Rs.5,000, discount 10% capped at Rs.700.

• Allocate discount proportionally by line, then calculate tax on discounted amounts.

Algorithm

1. Read product lines.

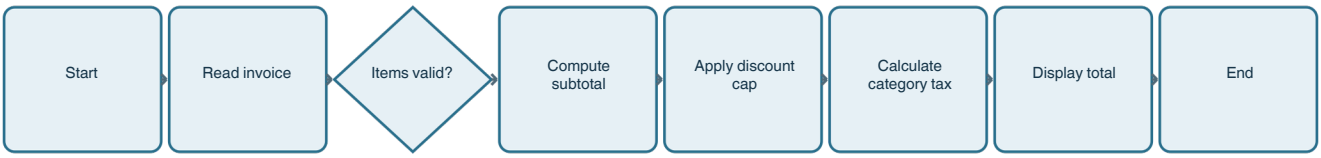
2. Validate category, price, and quantity.

3. Calculate subtotal and threshold discount subject to cap.

4. Allocate discount proportionally across lines.

5. Calculate taxes and final total; display invoice.

Flowchart



Pseudocode

```
START
READ invoice items
VALIDATE item data
subtotal = sum of line totals
IF subtotal >=5000: discount = MIN(10%, 700)
FOR each line: allocate discount and calculate category tax
DISPLAY invoice totals
END
```

Python source code

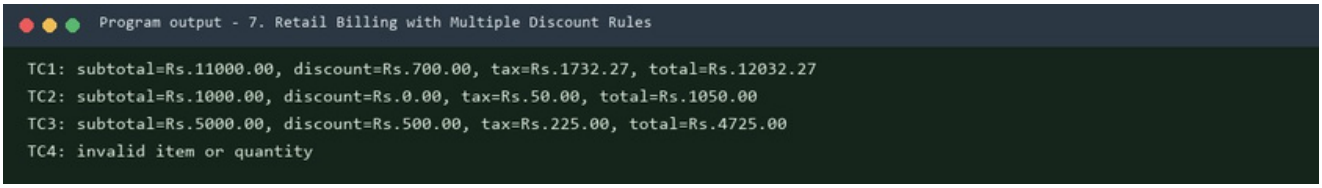
```
scenarios = [
    [("Rice","grocery",100,10),("Phone","electronics",10000,1)],
    [("Milk","grocery",100,10)], [("Notebook","grocery",100,50)],
    [("Pen","grocery",20,0)]]
for number, items in enumerate(scenarios, 1):
    print("TC"  +  str(number)  +  ":",  end="  ")
    valid = True
    subtotal = 0
    for name, category, price, quantity in items:
        if category not in ("grocery","electronics") or price < 0 or quantity <= 0:
            valid = False
        subtotal += price * quantity
    if not valid:
        print("invalid item or quantity")
    else:
        discount = min(subtotal * 0.10, 700) if subtotal >= 5000 else 0
        tax = 0
        for name, category, price, quantity in items:
            line = price * quantity
            taxable = line - discount * line / subtotal if subtotal else line
            rate = 0.05 if category == "grocery" else 0.18
            tax += taxable * rate
```

```
        print("subtotal=Rs." + format(subtotal, ".2f") + ", discount=Rs." +
              format(discount, ".2f") + ", tax=Rs." + format(tax, ".2f") +
              ", total=Rs." + format(subtotal-discount+tax, ".2f"))
```

Test cases

Test	Input	Expected result
TC1	Rice grocery Rs.100 x10; phone electronics Rs.10,000 x1.	Subtotal Rs.11,000; discount Rs.700; tax Rs.1,732.27; total Rs.12,032.27.
TC2	Milk grocery Rs.100 x10.	Subtotal Rs.1,000; no discount; tax Rs.50; total Rs.1,050.
TC3	Notebook grocery Rs.100 x50.	Subtotal Rs.5,000; discount Rs.500; tax Rs.225; total Rs.4,725.
TC4	Pen Rs.20 quantity 0.	Rejected: quantity must be positive.

Captured program output



Conclusion

The invoice applies discount thresholds and caps before category taxes. Proportional allocation keeps multi-category taxation consistent.

5. Attendance Recovery Computation

Problem statement

Calculate current attendance and the minimum number of consecutive attended classes needed to reach an eligibility threshold.

Objectives

• Calculate current attendance percentage.

• Find the minimum future attended classes needed to meet a target.

• Handle already-eligible records and invalid inputs.

Assumptions

• All future classes are attended. Find the smallest n such that (attended+n)/(conducted+n) >= target.

• If already eligible, required classes are zero.

• Conducted classes must be positive and attended cannot exceed conducted.

Algorithm

1. Read attended, conducted, and target values.

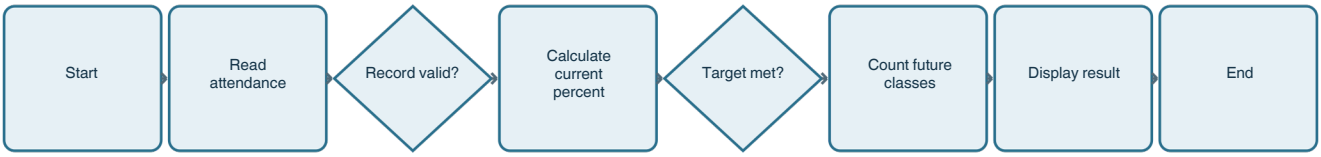
2. Validate counts and percentage.

3. Calculate current attendance.

4. Start at zero future classes; increment until the target is reached.

5. Display percentage and minimum count.

Flowchart



Pseudocode

```
START
READ attendance record
IF conducted <=0 or attended>conducted: invalid
current = attended/conducted x 100
n = 0
WHILE future percentage < target: increment n
DISPLAY current and n
END
```

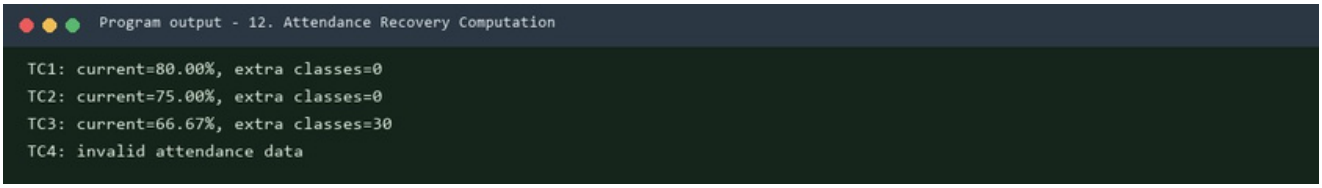
Python source code

```
records = [(72,90,75),(60,80,75),(60,90,75),(10,0,75)]
for number, (attended, conducted, target) in enumerate(records, 1):
    print("TC" + str(number) + ":", end=" ")
    if conducted <= 0 or attended < 0 or attended > conducted or not 0 <= target <= 100:
        print("invalid attendance data")
    else:
        current = attended / conducted * 100
        extra = 0
        while (attended + extra) / (conducted + extra) * 100 < target:
            extra += 1
        print("current=" + format(current, ".2f") + "%, extra classes=" + str(extra))
```

Test cases

Test	Input	Expected result
TC1	72 of 90; target 75%.	Current 80%; zero extra classes.
TC2	60 of 80; target 75%.	Current 75%; zero extra classes.
TC3	60 of 90; target 75%.	30 extra classes.
TC4	10 attended of 0 conducted.	Rejected: conducted classes must be positive.

Captured program output



Conclusion

The loop identifies the minimum number of future attended classes exactly, including already-eligible students and invalid records.

6. Warehouse Inventory Reconciliation

Problem statement

Reconcile opening stock, received and dispatched quantities, and recorded closing stock; detect discrepancies and calculate replenishment.

Objectives

• Calculate expected closing quantities.

• Detect differences from recorded closing stock.

• Calculate units needed to reach a target stock level.

Assumptions

• Expected closing = opening + received - dispatched.

• Dispatch cannot exceed available stock; quantities are nonnegative whole units.

• Discrepancy = recorded - expected; replenish = max(0, target - recorded).

Algorithm

1. Read each product record.

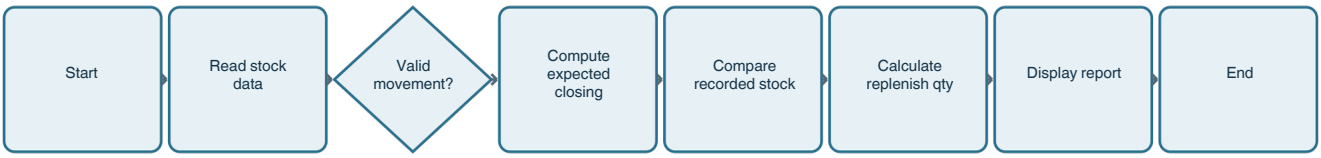
2. Validate quantities and available stock.

3. Calculate expected closing and discrepancy.

4. Calculate replenishment to target.

5. Display results or validation error.

Flowchart



Pseudocode

```
START
FOR each product record
IF any quantity negative or dispatched>available: invalid
expected = opening + received - dispatched
discrepancy = recorded - expected
replenish = MAX(0,target-recorded)
DISPLAY reconciliation
END
```

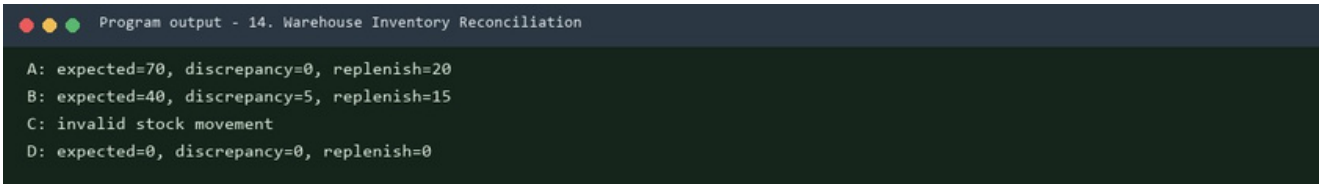
Python source code

```
records=[("A",100,40,70,70,90),("B",50,20,30,45,60),("C",10,0,11,0,5),("D",0,25,25,0,0)]
for product, opening, received, sent, recorded, target in records:
    print(product + ":", end=" ")
    if min(opening,received,sent,recorded,target) < 0 or sent > opening+received:
        print("invalid stock movement")
    else:
        expected=opening+received-sent
        print("expected="+str(expected)+", discrepancy="+str(recorded-expected)+
              ", replenish="+str(max(0,target-recorded)))
```

Test cases

Test	Input	Expected result
TC1	Opening 100, received 40, dispatched 70, recorded 70, target 90.	Expected 70; discrepancy 0; replenish 20.
TC2	Opening 50, received 20, dispatched 30, recorded 45, target 60.	Expected 40; discrepancy +5; replenish 15.
TC3	Opening 10, received 0, dispatched 11.	Rejected: dispatch exceeds available stock.
TC4	Opening 0, received 25, dispatched 25, recorded 0, target 0.	Expected 0; discrepancy 0; replenish 0.

Captured program output



Conclusion

Reconciliation separates a discrepancy from the replenishment need, supporting accurate stock correction and purchasing.

7. Student Marks Matrix Analysis

Problem statement

Analyze student marks across subjects; calculate student totals and averages, identify top performers, and flag low-average subjects.

Objectives

• Calculate row-wise student totals and averages.

• Calculate column-wise subject averages using nested lists.

• Identify top students and subjects with averages below 50.

Assumptions

• Rows represent students; columns represent subjects. Marks are 0..100.

• All rows must have equal length. A subject is flagged if its average is below 50.

• Tied highest totals share top rank.

Algorithm

1. Read the marks matrix.

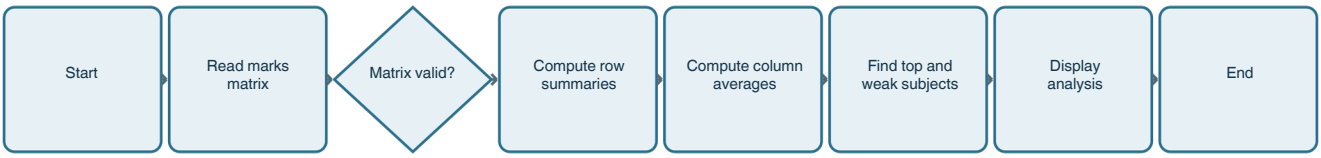
2. Validate the matrix shape and mark range.

3. Calculate each student total and average.

4. Calculate each subject average.

5. Display top student rows and weak subjects.

Flowchart



Pseudocode

```
START
READ marks matrix
VALIDATE shape and marks
FOR each row: calculate total and average
FOR each column: calculate average
FIND max total and averages <50
DISPLAY summaries
END
```

Python source code

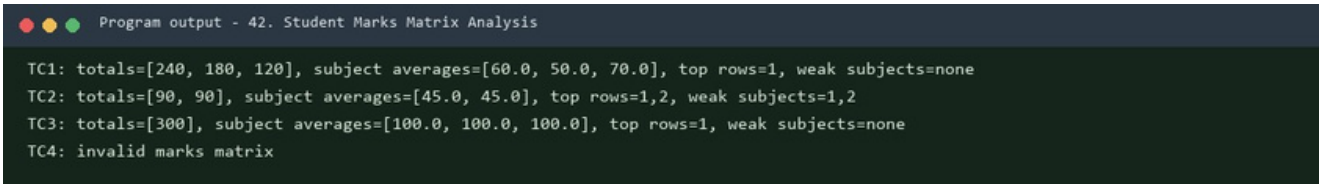
```
matrices=[[[80,70,90],[60,50,70],[40,30,50]],[[40,50],[50,40]],[[100,100,100]],[[80,101]]]
for number, marks in enumerate(matrices,1):
print("TC"+str(number)+":",end=" ")
cols=len(marks[0]) if marks else 0
valid=cols>0 and all(len(row)==cols for row in marks)
for row in marks:
        for mark in row:
            if mark<0 or mark>100: valid=False
    if not valid:
        print("invalid marks matrix")
    else:
        totals=[sum(row) for row in marks]
        averages=[]
        for col in range(cols):
            column_total=0
            for row in marks: column_total+=row[col]
            averages.append(column_total/len(marks))
        tops=[]
        for row_no in range(len(totals)):
            if totals[row_no]==max(totals): tops.append(str(row_no+1))
        weak=[]
        for col in range(len(averages)):
```

```
            if averages[col]<50: weak.append(str(col+1))
        print("totals="+str(totals)+", subject averages="+str(averages)+
              ",top rows="+",".join(tops)+", weak subjects="+(",".join(weak) if weak else "none"))
```

Test cases

Test	Input	Expected result
TC1	[[80,70,90],[60,50,70],[40,30,50]]	Totals 240,180,120; row 1 top; subject averages 60,50,70; none flagged.
TC2	[[40,50],[50,40]]	Both total 90 and jointly top; both subject averages 45 and flagged.
TC3	[[100,100,100]]	Total 300 and average 100; no weak subject.
TC4	[[80,101]]	Rejected: marks must be 0..100.

Captured program output



Conclusion

Nested-list processing produces student-level and subject-level summaries. Matrix validation prevents malformed input from producing misleading results.

8. Sports Tournament Ranking System

Problem statement

Store match results, calculate team points and goal differences, and rank teams under a stated competition policy.

Objectives

• Award points for wins, draws, and losses.

• Calculate goal difference from match scores.

• Rank teams deterministically with tie-break rules.

Assumptions

• Win=3 points, draw=1, loss=0; rank by points descending, goal difference descending, then name alphabetically.

• Scores are nonnegative integers and teams in a match must be distinct.

Algorithm

1. Read match results.

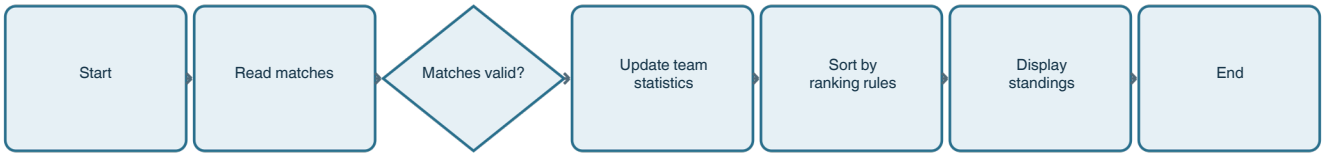
2. Initialize each team with zero points and goal difference.

3. Validate and process each score, updating both teams.

4. Order teams using the ranking policy.

5. Display standings or reject invalid match data.

Flowchart



Pseudocode

```
START
READ tournament matches
VALIDATE scores and team names
FOR each match: update points and goal difference
ORDER by points DESC, GD DESC, name ASC
DISPLAY rankings
END
```

Python source code

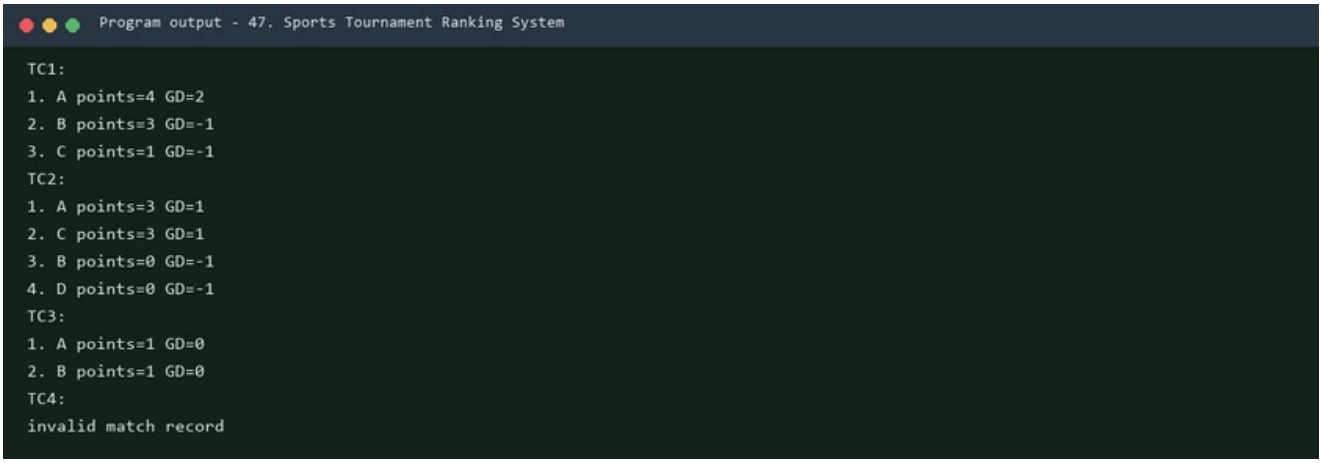
```
tournaments=[[("A","B",2,0),("B","C",1,0),("A","C",1,1)],[("A","B",1,0),("C","D",2,1)],[("A","B",0,0)],[("A","A",2,1)]]
for number,matches in enumerate(tournaments,1):
    teams=[]
    valid=True
    for a,b,sa,sb in matches:
        if a==b or sa<0 or sb<0: valid=False
        for team in (a,b):
            found=False
            for entry in teams:
                if entry[0]==team: found=True
            if not found: teams.append([team,0,0])
    print("TC"+str(number)+":")
    if not valid: print("invalid match record")
    else:
        for a,b,sa,sb in matches:
            for entry in teams:
                if entry[0]==a:
                    entry[2]+=sa-sb
                    if sa>sb: entry[1]+=3
                    elif sa==sb: entry[1]+=1
                if entry[0]==b:
                    entry[2]+=sb-sa
                    if sb>sa: entry[1]+=3
                    elif sa==sb: entry[1]+=1
        for i in range(len(teams)):
            for j in range(i+1,len(teams)):
                x,y=teams[i],teams[j]
```

```
            ify[1]>x[1] or (y[1]==x[1] and y[2]>x[2]) or (y[1]==x[1] and y[2]==x[2] and y[0]<x[0]):
                teams[i],teams[j]=teams[j],teams[i]
    for rank,entry in enumerate(teams,1):
        print(str(rank)+". "+entry[0]+" points="+str(entry[1])+" GD="+str(entry[2]))
```

Test cases

Test	Input	Expected result
TC1	A-B 2-0; B-C 1-0; A-C 1-1.	A:4 points/+2 GD; B:3/-1; C:1/-1. Rank A,B,C.
TC2	A-B 1-0; C-D 2-1.	A and C tie on points and GD; alphabetical tie-break ranks A first.
TC3	A-B 0-0.	Both earn 1 point and zero GD; A ranks first alphabetically.
TC4	A-A 2-1.	Rejected: a team cannot play itself.

Captured program output



Conclusion

Explicit point and tie-break policies make tournament standings transparent and repeatable, including tied teams.

9. Smart Canteen Order and Stock Management

Problem statement

Process successive canteen orders, calculate bills, update stock, reject unavailable quantities, and summarize sales and remaining inventory.

Objectives

• Validate orders against menu items and current stock.

• Calculate sales only for accepted orders.

• Maintain inventory after each successful order.

Assumptions

• Menu: idli Rs.30, stock 10; dosa Rs.50, stock 5. No tax or discounts.

• Quantity must be a positive whole number; each test begins with fresh stock.

• Rejected orders do not change stock or sales.

Algorithm

1. Initialize menu and stock.

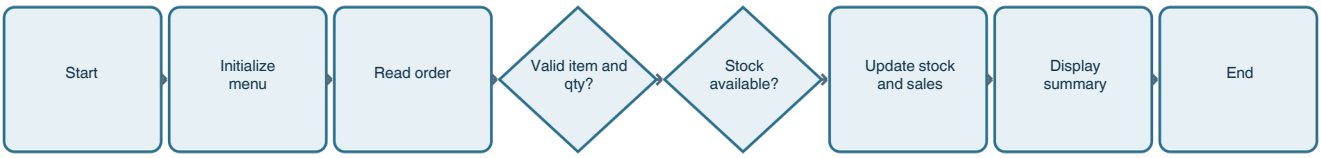
2. For each test scenario, reset stock and sales.

3. Validate item, quantity, and availability.

4. Accept valid orders and update stock and sales.

5. Display order decisions and final totals.

Flowchart



Pseudocode

```
START
INITIALIZE menu and stock
READ each order
IF item or quantity invalid: reject
ELSE IF quantity exceeds stock: reject
ELSE update stock and sales
DISPLAY daily summary
END
```

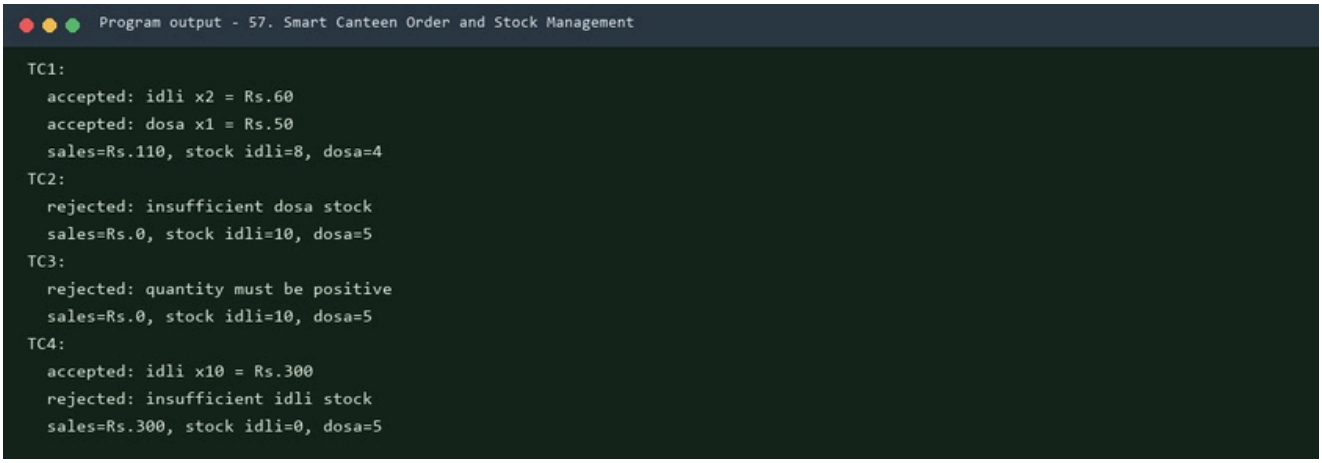
Python source code

```
scenarios=[[("idli",2),("dosa",1)],[("dosa",6)],[("idli",0)],[("idli",10),("idli",1)]]
for number,orders in enumerate(scenarios,1):
stock=[["idli",30,10],["dosa",50,5]]
sales=0
print("TC"+str(number)+":")
for item,qty in orders:
        found=False
        for entry in stock:
            if entry[0]==item:
            found=True
            if qty<=0 or qty!=int(qty): print("      rejected: quantity must be positive")
                elif qty>entry[2]: print(" rejected: insufficient "+item+" stock")
                else:
                    cost=qty*entry[1]
                    entry[2]-=qty
                    sales+=cost
                    print(" accepted: "+item+" x"+str(qty)+" = Rs."+str(cost))
        if not found: print("  rejected: item not on menu")
    print("  sales=Rs."+str(sales)+", stock idli="+str(stock[0][2])+", dosa="+str(stock[1][2]))
```

Test cases

Test	Input	Expected result
TC1	2 idli then 1 dosa.	Both accepted; sales Rs.110; idli stock 8; dosa stock 4.
TC2	6 dosas.	Rejected: only 5 are available; stock and sales unchanged.
TC3	0 idli.	Rejected: quantity must be positive.
TC4	10 idli then 1 idli.	First accepted for Rs.300; second rejected; idli stock 0; sales Rs.300.

Captured program output



Conclusion

Checking stock before updating it prevents overselling. Sales totals include accepted orders only, keeping inventory and revenue consistent.

10. Student Academic Risk Classification

Problem statement

Combine marks, attendance, and laboratory scores to classify academic risk and identify simultaneous risk factors.

Objectives

• Evaluate three independent risk indicators.

• Assign low, medium, or high risk consistently.

• Reject percentages outside the valid range.

Assumptions

• Risk factors: marks <50, attendance <75%, lab score <50%.

• High risk >=2 factors, medium risk exactly 1, low risk none.

• All input scores are between 0 and 100.

Algorithm

1. Read marks, attendance, and lab score.

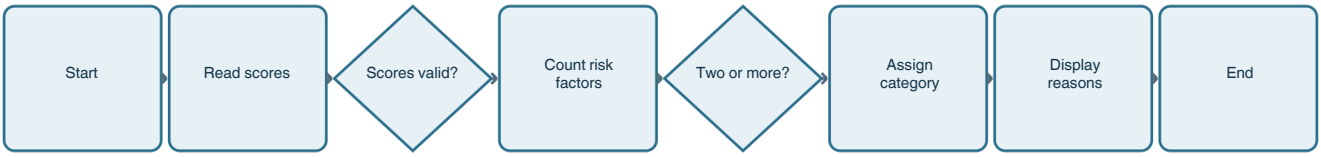
2. Validate each value is within 0..100.

3. Count how many risk rules apply.

4. Classify using the number of factors.

5. Display category and reasons.

Flowchart



Pseudocode

```
START
READ student scores
IF any score outside 0..100: invalid
COUNT low-mark, low-attendance, low-lab conditions
IF count>=2: HIGH; ELSE IF count==1: MEDIUM; ELSE LOW
DISPLAY category and factors
END
```

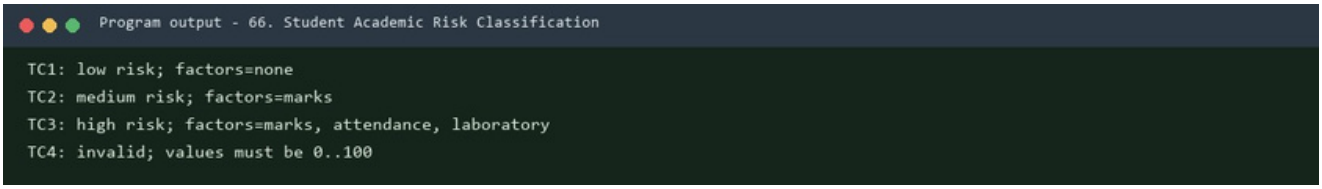
Python source code

```
students=[("TC1",70,80,60),("TC2",49,80,60),("TC3",40,70,45),("TC4",101,90,80)]
for name,marks,attendance,lab in students:
    print(name+":",end=" ")
    if min(marks,attendance,lab)<0 or max(marks,attendance,lab)>100:
        print("invalid; values must be 0..100")
    else:
        factors=[]
        if marks<50: factors.append("marks")
        if attendance<75: factors.append("attendance")
        if lab<50: factors.append("laboratory")
        if len(factors)>=2: risk="high"
        elif len(factors)==1: risk="medium"
        else: risk="low"
        print(risk+" risk; factors="+( ", ".join(factors) if factors else "none"))
```

Test cases

Test	Input	Expected result
TC1	Marks 70, attendance 80%, lab 60%.	0 factors; low risk.
TC2	Marks 49, attendance 80%, lab 60%.	1 factor; medium risk.
TC3	Marks 40, attendance 70%, lab 45%.	3 factors; high risk.
TC4	Marks 101, attendance 90%, lab 80%.	Rejected: values must be within 0..100.

Captured program output



Conclusion

Combining academic and attendance indicators helps identify students who may need support and explains each risk classification.